
ShoppingApp — Complete Function Documentation

Project layout (quick reference)

```
ShoppingApp/
├── CMakeLists.txt
├── main.cpp
├── data/
│   ├── carts/
│   ├── Items.txt
│   ├── Transactions.txt
│   └── Users.txt
├── include/
│   ├── shopping_app.h
│   ├── menu/
│   │   ├── main_menu.h
│   │   ├── admin/
│   │   │   ├── admin_menu.h
│   │   │   └── actions/
│   │   │       └── admin_actions.h
│   │   └── user/
│   │       ├── buyer_menu.h
│   │       ├── seller_menu.h
│   │       ├── user_menu.h
│   │       └── actions/
│   │           ├── buyer_actions.h
│   │           ├── selller_actions.h
│   │           └── user_actions.h
│   ├── models/
│   │   ├── cart.h
│   │   ├── item.h
│   │   ├── transaction.h
│   │   └── user.h
│   ├── services/
│   │   └── file_handling.h
│   └── utils/
│       └── utils.h
└── src/...
```

Global conventions / data formats

- All persistent files are **text**. Expected files:
 - data/Users.txt — users saved in repeated blocks:
 - <userID> <password>
 - <name>
 - <address>
 - <contact number>
 -
 - (repeat)
 - data/Items.txt — items saved as repeated blocks:

- o <productID> <sellerID>
 - o <item name>
 - o <category>
 - o <description>
 - o <quantity> <unit price>
 - o
 - o (repeat)
 - o data/Transactions.txt — transactions appended in a human-readable block format (implementation-specific but consistent between save/load).
 - o data/carts/<userID>.bag (or .txt if you change the extension) — saved per-user cart in text; each line or block records <productID> <sellerID> <quantity> <unitPrice> <itemName...> (the exact format is implemented in FileHandling::LoadCartText / SaveCartText).
 - Program tolerates missing files (loads empty collections); saving requires directories to exist (data/ and data/carts/). main.cpp ensures directories exist (best-effort) before I/O.
 - Many displays print tables. Utils::PrintItemsTable, AdminActions::ShowAllUsers and ShowAllSellers compute dynamic column widths by scanning contents.
-

File: main.cpp

```
int main()
```

Prototype

```
int main();
```

Purpose

Program entry point. Ensures data directories exist, loads Users/Items/Transactions into memory, runs the MainMenu loop, and saves data on exit.

Behavior / Steps

1. Ensure directories data and data/carts exist (best-effort). Prints warnings if directory creation fails but continues.
2. Constructs a ShoppingApp instance (app).
3. Attempts to load:
 - o data/Users.txt via app.LoadUsersFromFile
 - o data/Items.txt via app.LoadItemsFromFile
 - o data/Transactions.txt via app.LoadTransactionsFromFile
 Each load is best-effort — if the file does not exist or cannot be parsed, the program prints a notice and continues with empty in-memory containers.
4. Constructs MainMenu menu(app); and calls menu.Show() — the main interactive loop.
5. On exit from the menu, saves:
 - o app.SaveUsersToFile("data/Users.txt")
 - o app.SaveItemsToFile("data/Items.txt")

- o `app.SaveTransactionsToFile("data/Transactions.txt")`

Each save attempt prints success or error to stdout/stderr.

6. Returns 0.

Notes

- Implemented to be beginner-friendly and cross-platform for directory creation: uses `_mkdir` on Windows and `mkdir` on POSIX. This avoids failed saves due to missing directories.
- The code does not require admin privileges for directory creation (regular user directories under project path).
- If you want carts loaded at login rather than Buy Menu, move the cart-load call accordingly.

Module: `include/shopping_app.h` / `src/shopping_app.cpp` (application state manager)

The `ShoppingApp` class is the in-memory model for users, items, and transactions and exposes helper methods used by menus and actions.

Common public methods (as used in code):

```
ShoppingApp::ShoppingApp()
```

Prototype

```
ShoppingApp();
```

Purpose: default constructor. Initializes empty containers for users, items, transactions.

```
bool LoadUsersFromFile(const std::string& filepath)
```

Prototype

```
bool LoadUsersFromFile(const std::string& filepath);
```

Purpose: Populate `users_` by parsing `Users.txt`.

Parameters: `filepath` — path to `Users` file.

Return: `true` on success (file opened and parsed), `false` on I/O error or parse failure (caller treats as "no users loaded").

Behavior:

- Opens file, reads user blocks; if a block is malformed it's skipped (loader attempts to be resilient).
- Creates `User` objects and pushes into `users_`.
- Does not enforce uniqueness beyond what `RegisterUser` enforces; the loader simply reads whatever is in file.

```
bool SaveUsersToFile(const std::string& filepath)
```

Prototype

```
bool SaveUsersToFile(const std::string& filepath);
```

Purpose: Writes `users_` to `Users.txt` using the multi-line format described above.

Return: true on successful write, false on error.

```
bool LoadItemsFromFile(const std::string& filepath)
```

Prototype

```
bool LoadItemsFromFile(const std::string& filepath);
```

Purpose: Parse `Items.txt` into in-memory `items_` vector following file layout.

Return: true on success, false on error.

Notes: Must enforce per-item fields; invalid blocks are skipped.

```
bool SaveItemsToFile(const std::string& filepath)
```

Prototype

```
bool SaveItemsToFile(const std::string& filepath);
```

Purpose: Write current `items_` to `Items.txt` in the multi-line format.

```
bool LoadTransactionsFromFile(const std::string& filepath)
```

Prototype

```
bool LoadTransactionsFromFile(const std::string& filepath);
```

Purpose: Parse `Transactions.txt` into `transactions_` in-memory vector. Parsing must match the saved format.

```
bool SaveTransactionsToFile(const std::string& filepath)
```

Prototype

```
bool SaveTransactionsToFile(const std::string& filepath);
```

Purpose: Persist `transactions_` to `Transactions.txt`. Implementation writes whole set (often truncating and writing all transactions).

```
const std::vector<User>& GetUsers() const
```

Prototype

```
const std::vector<User>& GetUsers() const;
```

Purpose: Read-only access to loaded users.

```
std::vector<Item>& GetItems()
```

Prototype

```
std::vector<Item>& GetItems();
```

Purpose: Access to items (modifiable by menu actions).

```
const std::vector<Transaction>& GetTransactions() const
```

Prototype

```
const std::vector<Transaction>& GetTransactions() const;
```

```
bool RegisterUser(const User& user)
```

Prototype

```
bool RegisterUser(const User& user);
```

Purpose: Add a new user to `users_` with business rules:

- Maximum 100 users.
- Unique `userID`.

Return: `true` on success (added), `false` if user was rejected (duplicate ID or limit reached).

Side effects: modifies `users_`.

Implementation notes:

- If rejected, implementation prints a diagnostic to `std::cerr` (we added helpful debug messages earlier).
- Validation (password length, name length, etc.) is performed by the higher-level UI or can be added to `User::SetX` methods.

```
Item* FindItemByProductId(int productId)
```

Prototype

```
Item* FindItemByProductId(int productId);
```

Purpose: Return pointer to `Item` in `items_` matching `productId`, or `nullptr` if not found.

Use: Buyer flows use this to validate product existence.

```
bool AddTransaction(const Transaction& t)
```

Prototype

```
bool AddTransaction(const Transaction& t);
```

Purpose: Validate and append transaction to `transactions_`, and update `items_` quantities accordingly.

Return: `true` if transaction was accepted and applied (inventory updated), `false` otherwise.

Behavior:

- Verify all items in transaction exist and have sufficient stock.
 - Ensure buyer != seller.
 - If valid, reduce item quantities in `items_` and append `t` to `transactions_`.
 - Optionally calls `FileHandling::AppendTransactionText` to add to `Transactions.txt` or relies on `SaveTransactionsToFile` at exit.
-

Models

All model headers are in `include/models/`. Each class is documented below.

`include/models/user.h` — User class

Fields (private)

- `int userID_` — numeric ID, unique across users.
- `std::string password_` — up to 10 chars (enforce in setter/validation).
- `std::string name_` — up to 20 chars.
- `std::string address_` — up to 30 chars.
- `long long contactNumber_` — numeric contact.

Public API

`User()`

Default constructor: initializes fields to zero/empty.

```
User(int userID, const std::string& password, const std::string& name,
const std::string& address, long long contactNumber)
```

Overloaded constructor: set all fields.

```
int GetUserID() const
```

Return `userID`.

```
const std::string& GetPassword() const
```

Return password reference.

```
const std::string& GetName() const
```

Return name.

```
const std::string& GetAddress() const
```

Return address.

```
long long GetContactNumber() const
```

Return contact number.

```
void SetUserID(int id)
```

Assign `userID_`. Uniqueness must be enforced externally.

```
void SetPassword(const std::string& password)
```

Set password (implementation may check `password.size() <= 10`).

```
void SetName(const std::string& name)
```

Set name (may validate length ≤ 20).

```
void SetAddress(const std::string& address)
```

Set address (may validate length ≤ 30).

```
void SetContactNumber(long long number)
```

Set contact number. No formatting enforced at setter — validation may be done by UI (`IsValidContactNumber` helper could be added).

Notes

- `User` is a simple POD-like class. The class must not enforce uniqueness of `userID` by itself (that is `ShoppingApp`'s responsibility).

`include/models/item.h` — `Item` **class**

Fields (private)

- `int productID_`
- `std::string itemName_`
- `std::string category_`
- `std::string description_`
- `int quantity_`
- `double unitPrice_`
- `int sellerID_`

Public API

`Item()`

Default constructor — zero/empty initializers.

```
Item(int productID, const std::string& itemName, const std::string&
category, const std::string& description, int quantity, double unitPrice,
int sellerID)
```

Full initializer.

Accessors (const)

- `int GetProductID() const;`
- `const std::string& GetItemName() const;`
- `const std::string& GetCategory() const;`
- `const std::string& GetDescription() const;`
- `int GetQuantity() const;`
- `double GetUnitPrice() const;`
- `int GetSellerID() const;`

Mutators

- `void SetProductID(int id);`
- `void SetItemName(const std::string& name);` (validate ≤ 20 characters if desired)
- `void SetCategory(const std::string& category);` (≤ 15)
- `void SetDescription(const std::string& desc);` (≤ 30)
- `void SetQuantity(int qty);` (ensure non-negative)
- `void SetUnitPrice(double price);` (non-negative)
- `void SetSellerID(int id);`

Notes

- `ShoppingApp` should ensure `productID` uniqueness across all items.
- Seller limit (max 20 different items per seller) is enforced in the seller add-item action logic (not by `Item` itself).

`include/models/cart.h` — `Cart` **and** `CartItem`

`struct CartItem`

Fields:

- `int productID;`
- `int sellerID;`
- `int quantity;`
- `double unitPrice;` // snapshot at time of add

- `std::string itemName; // snapshot for display`

Constructors:

- `CartItem();`
- `CartItem(int productID, int sellerID, int quantity, double unitPrice, const std::string& itemName);`

class Cart

Private:

- `std::vector<CartItem> items_;` — each distinct product is an entry.

Public:

- `Cart();` default
- `Cart(const std::vector<CartItem>& items);` overloaded
- `const std::vector<CartItem>& GetItems() const;` read-only access
- `bool AddItem(const CartItem& item);` Adds or merges; should enforce max distinct items (10). Expected behavior:
 - If same product exists, either increase quantity or replace depending on design. Our actions code expects `AddItem` to merge by increasing quantity or reject if cart full / quantity invalid.
 - Returns `true` on success, `false` on failure (e.g., cart full).
- `bool RemoveItemByProductId(int productId);` Remove single product by productId.
- `bool RemoveItemsBySellerId(int sellerId);` Remove all items with sellerId; return `true` if any removed.
- `bool UpdateQuantity(int productId, int newQuantity);` Set new quantity (≥ 0).
- `void Clear();` Remove all items.
- `int Size() const;` Number of distinct items.

Notes

- Cart has a per-user maximum of 10 distinct items; this is enforced in `AddItem`.
- Cart save/load lives in `FileHandling`.

`include/models/transaction.h` — `Transaction`, `TransactionItem`, `Date`

struct Date

Fields: `int month, day, year;`

Constructors:

- `Date();`
- `Date(int m, int d, int y);`

struct TransactionItem

Fields:

- int productID;
- std::string itemName;
- int quantity;
- double unitPrice;

Constructors:

- TransactionItem();
- TransactionItem(int productID, const std::string& itemName, int quantity, double unitPrice);

class Transaction

Private:

- Date date_;
- std::vector<TransactionItem> items_; (max 5 enforced externally)
- int buyerID_;
- int sellerID_;
- double totalAmount_; (sum of quantity * unitPrice for items)

Public:

- Transaction();
- Transaction(const Date& date, const std::vector<TransactionItem>& items, int buyerID, int sellerID, double totalAmount);
- const Date& GetDate() const;
- const std::vector<TransactionItem>& GetItems() const;
- int GetBuyerID() const;
- int GetSellerID() const;
- double GetTotalAmount() const;
- void SetDate(const Date& date);
- void SetItems(const std::vector<TransactionItem>& items); (should validate max 5)
- void SetBuyerID(int id);
- void SetSellerID(int id);
- void SetTotalAmount(double amount);
- bool AddItem(const TransactionItem& item); returns false if adding would exceed 5 items.

Notes

- Transaction creation utilities exist in BuyerActions (e.g., CreateTransactionFromCartItems) to convert cart items into transactions and compute totals.

Services

`include/services/file_handling.h` — FileHandling static helpers

Class overview

Static helper functions for reading/writing the project text formats (Users, Items, Transactions, Carts).

Public static methods

`FileHandling()`

Default constructor — no instance state.

```
static bool LoadUsersText(const std::string& filepath, std::vector<User>& outUsers)
```

Purpose: Parse `Users.txt` into `outUsers`.

Parameters: `filepath` — path to file; `outUsers` — vector to populate.

Return: `true` if file read and parsed (even with some skipped blocks), `false` on I/O failure.

Behavior:

- Reads the file in blocks (one user per block). If malformed blocks occur the loader will try to skip them and continue.
- Use this in `main.cpp`: `app.LoadUsersFromFile(...)` typically wraps this.

```
static bool SaveUsersText(const std::string& filepath, const std::vector<User>& users)
```

Purpose: Save `users` to file in multi-line block format. Returns `true` on success.

```
static bool LoadItemsText(const std::string& filepath, std::vector<Item>& outItems)
```

Purpose: Read `Items.txt` to populate items vector. Returns `true` on success.

```
static bool SaveItemsText(const std::string& filepath, const std::vector<Item>& items)
```

Purpose: Write items vector to file.

```
static bool AppendTransactionText(const std::string& filepath, const Transaction& transaction)
```

Purpose: Append a single transaction (human-readable format) to `Transactions.txt`. Returns `true` on success.

Notes: If you prefer accumulating and writing all transactions at once, you can use `SaveTransactionsText` (or `SaveTransactionsToFile` in the app).

```
static bool LoadTransactionsText(const std::string& filepath, std::vector<Transaction>& outTransactions)
```

Purpose: Parse `Transactions.txt` into `outTransactions`.

```
static bool LoadCartText(const std::string& filepath, Cart& outCart)
```

Purpose: Load a saved cart from `data/carts/<userId>.bag`. Returns `true` on success.

Behavior: Parser expects each cart line or block describing `CartItem`s; it converts to `CartItem` and populates `outCart`. If file missing, loader returns `false` and caller treats cart as empty.

```
static bool SaveCartText(const std::string& filepath, const Cart& cart)
```

Purpose: Save `cart` to text file. Returns `true` on success.

Implementation notes

- All file I/O uses `std::ifstream/std::ofstream`. If directories are missing, writes will fail — `main.cpp` ensures directories exist.
- Loaders try to be resilient, skip malformed entries rather than crash.
- You can change cart extension from `.bag` to `.txt` by adjusting the `BuyerMenu` cart path construction.

Utilities

`include/utls/utls.h` / `src/utls/utls.cpp` — **Utls** class

Purpose

Presentation and string helpers used throughout the menus for printing aligned tables and doing case-insensitive searches.

Public static methods

```
static std::string RightJustify(const std::string& input, size_t width)
```

Purpose: Return `input` padded on the left with spaces to make total length `width`. If `input.size() >= width`, returns `input` unchanged.

```
static std::string LeftJustify(const std::string& input, size_t width)
```

Purpose: Return `input` padded on the right with spaces to `width`.

```
static void PrintItemsTable(const std::vector<Item>& items)
```

Purpose: Print an aligned table of items with dynamic column widths computed from contents.

Columns: `ProductID` (right), `Name` (left), `Category` (left), `UnitPrice` (right), `Qty` (right).

Behavior:

- Formats `UnitPrice` to two decimal places using `ostringstream` with `fixed` and `precision(2)`.

- Uses `COLUMN_SPACING` constant (in `utils.cpp`) to control inter-column spaces (edit the constant to your preference).
- Prints header, separator line, then all item rows.

```
static bool ContainsIgnoreCase(const std::string& source, const
std::string& searchTerm)
```

Purpose: Returns `true` if `searchTerm` exists in `source` case-insensitively.

Behavior: Builds lowercase copies of `source` and `searchTerm` and runs `find`. Empty `searchTerm` returns `true`.

Notes

- `COLUMN_SPACING` is declared in `utils.cpp`. You can change it to adjust gap between columns for all `Utils`-based tables.

Menus & Actions (UI layer)

These classes live under `include/menu/` and `src/menu/`. Each menu class has a `Show()` method that implements an interactive loop using `std::cin / std::cout`. Actions are separated into helper classes (e.g., `AdminActions`, `BuyerActions`, `SellerActions`, `UserActions`) to keep UI code small.

`include/menu/main_menu.h` — **MainMenu**

```
MainMenu::MainMenu(ShoppingApp& app)
```

Purpose: constructor holds reference to application state.

```
void MainMenu::Show()
```

Purpose: Runs top-level menu loop:

- 1: Register as a User
 - Prompts for `userID`, `password`, `name`, `address`, `contact`.
 - Constructs `User` and calls `app_.RegisterUser(u)`.
 - Prints success or failure message.
- 2: User Menu — constructs `UserMenu(app_)` and calls `Show()`.
- 3: Admin Menu — constructs `AdminMenu(app_)` and calls `Show()`.
- 4: Exit — break loop and return to `main`.

Notes

- Input is validated lightly. Improper numeric input clears `std::cin` and continues.

```
include/menu/user/user_menu.h — UserMenu
```

```
UserMenu::UserMenu(ShoppingApp& app)
```

Constructor stores `app` reference.

```
void UserMenu::Show()
```

Behavior:

1. Prompts for user ID and password.
2. Uses `UserActions::Login(app.GetUsers(), userId, password)` to validate credentials.
3. On login success:
 - o Looks up user's name in `app.GetUsers()` and greets the user: `Welcome, <name>!`
 - o Shows user menu:
 - 1: `SellerMenu` (with logged-in user ID)
 - 2: `BuyerMenu` (with logged-in user ID)
 - 3: Exit User Menu (returns to Main Menu)
4. On login failure: prints message and returns.

Notes

- The welcome message was added where we fetch name from `app.GetUsers()`.

```
include/menu/user/buyer_menu.h — BuyerMenu
```

```
BuyerMenu::BuyerMenu(ShoppingApp& app, int userId)
```

Stores reference to `app` and `loggedInUserId`.

```
void BuyerMenu::Show()
```

Purpose: Offer buy-related features. On entry attempts to load the user's cart file `data/carts/<userId>.bag` (via `FileHandling::LoadCartText`) and on exit saves it via `FileHandling::SaveCartText`.

Menu options and behaviors

- 1. View all products — uses `BuyerActions::ViewAllProducts(app_.GetItems())`. This prints sellers and tables of their items (we implemented printing with `Utils::PrintItemsTable`).
- 2. Show all products by specific seller — prompts `sellerId` and calls `BuyerActions::ViewProductsBySeller`.

- 3. Search products by category — prompts category and prints matching items (using `ContainsIgnoreCase`).
- 4. Search products by name — substring search using `ContainsIgnoreCase`.
- 5. Add to cart — prompts productId and quantity:
 - Validates the product exists via `app_.FindItemByProductId(productId)`.
 - Validates buyer isn't the seller.
 - Calls `BuyerActions::AddToCart(cart, item, qty)` — `AddToCart` should:
 - Check stock availability
 - Enforce max 10 distinct items in cart
 - Add or merge quantities in cart
 - On success prints confirmation.
- 6. Edit cart — submenu:
 - 1: Remove all items from seller — `cart.RemoveItemsBySellerId(sellerId)`
 - 2: Remove specific item — `cart.RemoveItemByProductId(productId)`
 - 3: Edit quantity — `BuyerActions::EditCartQuantity(cart, productId, newQty)` (updates by calling `cart.UpdateQuantity`)
 - 4: Finish Edit Cart — return to Buy Menu
- 7. Check out — checkout submenu:
 - Collect a date (month day year)
 - 1. All — split cart by seller and for each seller:
 - Build a Transaction from their cart items (via `BuyerActions::CreateTransactionFromCartItems(group, buyerId, sellerId, date)`)
 - Call `app_.AddTransaction(transaction)` to validate/apply transaction (updates items_ quantities and appends to `transactions_`)
 - If success remove corresponding items from `cart`.
 - 2. By specific seller — same as above but only that seller.
 - 3. Specific item — check out one cart item.
 - 4. Exit Checkout
- 8. Exit Buy Menu — saves cart file and returns to User Menu.

Files used

- Load: `data/carts/<userId>.bag` via `FileHandling::LoadCartText`.
- Save: `data/carts/<userId>.bag` via `FileHandling::SaveCartText`.

Notes

- The buyer flows rely on action helpers (`BuyerActions`) to implement logic such as `CreateTransactionFromCartItems`, `AddToCart`, `EditCartQuantity`.
- Real-world correctness checks (race conditions, concurrency) are not in scope for this single-threaded command-line app.

`include/menu/user/seller_menu.h` — `SellerMenu`

```
SellerMenu::SellerMenu(ShoppingApp& app, int userId)
```

Constructor — stores app and seller's user id.

```
void SellerMenu::Show()
```

Purpose: Seller-side features:

- 1. Add New Item — prompts product ID, item name, category, description, quantity, price, constructs Item with sellerID=loggedInUserId_ and calls SellerActions::AddNewItem(app.GetItems(), item, sellerId) which must:
 - Ensure productID uniqueness across all items
 - Ensure seller has not exceeded 20 different items
 - Return true on success
- 2. Edit Stock — shows seller's products then asks productID to edit; present edit submenu:
 - 1. Replenish — SellerActions::Replenish(*p, addQty) increments quantity
 - 2. Change Price — SellerActions::ChangePrice(*p, newPrice) updates unit price
 - 3. Change Item Name — SellerActions::ChangeItemName(*p, newName)
 - 4. Change Category — SellerActions::ChangeCategory
 - 5. Change Description — SellerActions::ChangeDescription
 - 6. Finish Editing
- 3. Show My Products — prints seller's items sorted by productID. The implementation uses a simple selection-sort to keep the code beginner-friendly and then calls Utils::PrintItemsTable to print.
- 4. Show My Low Stock Products — displays items where quantity < 5, one at a time; user navigates with N and X.
- 5. Exit Sell Menu — returns to User Menu.

Notes

- The seller menu relies on SellerActions for logic that manipulates items_ vector.
- Deleting an item is not implemented in the UI; manual edit of data/Items.txt is currently required or a new action would need to be added.

```
include/menu/admin/admin_menu.h — AdminMenu
```

```
AdminMenu::AdminMenu(ShoppingApp& app)
```

Constructor: stores app reference and default AdminActions.

```
void AdminMenu::Show()
```

Behavior:

1. Prompts admin password. Uses `AdminActions::Authenticate(password)`, which by default compares to "H3LL0?".
2. If authentication passes, shows admin options:
 - o 1. Show All Users — `actions_.ShowAllUsers(app_.GetUsers())` prints a dynamic-width table (UserID, Password, Name, Address, Contact).
 - o 2. Show All Sellers — `actions_.ShowAllSellers(app_.GetUsers(), app_.GetItems())` prints a dynamic table with NumItems column (computed by scanning items).
 - o 3. Show Total Sales in Given Duration — prompts start/end date; calls `actions_.ShowTotalSalesInDuration(app_.GetTransactions(), ...)` returning double sum.
 - o 4. Show Sellers Sales in Given Duration — `actions_.ShowSellersSales(...)` aggregates total sales per seller in range and prints table.
 - o 5. Show Shopaholics in Given Duration — `actions_.ShowShopaholics(...)` aggregates totals per buyer in range and prints table.
 - o 6. Back to Main Menu — returns to main menu.

Notes

- Table printing uses dynamic column widths; inter-column spacing controlled by `COLUMN_SPACING` constant in `admin_actions.cpp`.
- Admin delete operations (delete user, delete transaction) are not in UI by default; we can add if requested.

Actions (helpers used by menus)

The "Actions" classes contain the business logic called by menus. Below are the important methods implemented or expected.

AdminActions

- `AdminActions() / AdminActions(const std::string& adminPassword)`
 - o Constructor; sets admin password string used by `Authenticate`.
- `bool Authenticate(const std::string& password) const`
 - o Compares to stored admin password and returns true if match.
- `void ShowAllUsers(const std::vector<User>& users) const`
 - o Prints aligned table of users. Uses `Utils::LeftJustify / RightJustify` to compute column widths from the data and prints header+separator+rows.
- `void ShowAllSellers(const std::vector<User>& users, const std::vector<Item>& items) const`
 - o Computes number of items per seller by scanning items, selects users who are sellers, and prints aligned table including NumItems.

- `double ShowTotalSalesInDuration(const std::vector<Transaction>& transactions, int sm, int sd, int sy, int em, int ed, int ey) const`
 - Returns the sum of transaction totals for those with date in inclusive range [start, end].
- `void ShowSellersSales(const std::vector<Transaction>& transactions, int sm, int sd, int sy, int em, int ed, int ey) const`
 - Aggregates total sales per seller in date range and prints two-column table SellerID | TotalSales.
- `void ShowShopaholics(const std::vector<Transaction>& transactions, int sm, int sd, int sy, int em, int ed, int ey) const`
 - Aggregates total amount bought per buyer in date range and prints BuyerID | TotalBought.

Notes

- Date range comparisons use a `DateInRange` helper which compares year, month, day lexicographically to check inclusive membership. No advanced calendar checks (e.g., month/day validation) are performed here.

BuyerActions (summary of expected/used helpers)

Important helpers used by `BuyerMenu` (some implemented in `src/menu/user/actions`):

- `bool ViewAllProducts(const std::vector<Item>& items)`
 - Prints all products grouped by seller. For each seller: print "Seller ID: " then item table via `Utils::PrintItemsTable`.
- `void ViewProductsBySeller(const std::vector<Item>& items, int sellerId)`
 - Filters items for `sellerId`, sorts by `productId`, then prints table.
- `std::vector<Item> SearchProductsByCategory(const std::vector<Item>& items, const std::string& category)`
 - Returns vector of items where `Utils::ContainsIgnoreCase(item.GetCategory(), category)` is true.
- `std::vector<Item> SearchProductsByName(const std::vector<Item>& items, const std::string& keyword)`
 - Returns vector of items where `ContainsIgnoreCase(item.GetItemName(), keyword)` is true.
- `bool AddToCart(Cart& cart, const Item& item, int qty)`
 - Business rules:
 - Validate `qty > 0` and `qty <= item.GetQuantity()`.
 - If `cart.Size() >= 10` and adding a new distinct product, reject.
 - If product already in cart, increase quantity (but validate max allowed per business rules).
 - On success, add a `CartItem(productId, sellerID, qty, item.GetUnitPrice(), item.GetItemName())` to cart.
 - Return true on success, false on failure.
- `bool EditCartQuantity(Cart &cart, int productId, int newQty)`

- o Locate product in cart and update quantity using `cart.UpdateQuantity(productId, newQty).`
- Transaction `CreateTransactionFromCartItems(const std::vector<CartItem>& group, int buyerId, int sellerId, const Date& date)`
 - o Convert `CartItems` to `TransactionItems` and compute `totalAmount`. Ensures all items belong to same seller and buyer != seller.

Notes

- Implementation details (merging quantities, maximum cart entries, transaction item limit 5) must be followed.
-

SellerActions (summary)

- `bool AddNewItem(std::vector<Item>& items, const Item& newItem, int sellerId)`
 - o Checks:
 - Product ID uniqueness (`productId` doesn't exist among `items`).
 - Seller hasn't exceeded 20 unique items.
 - o On success, `items.push_back(newItem)` and return `true`.
- `Item* FindSellerItem(std::vector<Item>& items, int sellerId, int productId)`
 - o Returns pointer to `Item` matching `sellerId` and `productId`, else `nullptr`.
- `bool Replenish(Item& item, int addQty)`
 - o `item.SetQuantity(item.GetQuantity() + addQty)` and returns `true` on success (no negative inputs).
- `bool ChangePrice(Item& item, double newPrice)`
 - o If `newPrice >= 0` set and return `true`.
- `void ChangeItemName(Item& item, const std::string& newName) etc.`

Notes

- Seller actions mutate `items` in-place; `ShoppingApp` will persist items when `SaveItemsToFile` is called.
-

Example flows and where each operation is handled

- **Register user** — done in `MainMenu::Show()` by collecting fields and calling `app.RegisterUser(user).`
- **Login** — `UserMenu::Show()` obtains user ID and password and checks with `UserActions::Login over app.GetUsers()`.

- **Add item (seller)** — `SellerMenu::Show()` collects item info and calls `SellerActions::AddNewItem(app.GetItems(), item, sellerId)`.
 - **View products** — `BuyerMenu::Show()` uses `BuyerActions::ViewAllProducts` and `Utils::PrintItemsTable`.
 - **Add to cart** — `BuyerMenu::Show()` calls `BuyerActions::AddToCart(cart, *item, qty)` which uses `Cart::AddItem`.
 - **Save/Load cart** — `BuyerMenu::Show()` at entry calls `FileHandling::LoadCartText` and on exit calls `FileHandling::SaveCartText`.
 - **Checkout** — `BuyerMenu` calls `BuyerActions::CreateTransactionFromCartItems` to build `Transactions` and `ShoppingApp::AddTransaction` to validate and apply them, and then `FileHandling::AppendTransactionText` or `SaveTransactionsToFile` persists them.
-

File & function-specific implementation notes & caveats

1. **Directories**
 - `main.cpp` attempts to create `data` and `data/carts`. If creation fails (permissions etc.) saving files will fail. The program prints warnings but continues.
 2. **File parsing resilience**
 - `FileHandling` loaders attempt to be robust (skip malformed blocks rather than abort). They will still produce partial results in the presence of tampering.
 3. **Field length validations**
 - Headers document limits (password ≤ 10 , name ≤ 20 , address ≤ 30 , itemName ≤ 20 , category ≤ 15 , description ≤ 30). These checks are not always enforced automatically in the basic setters; UI-level validation is recommended (e.g., on registration and add item).
 4. **Numeric validations**
 - Contact number format is not enforced strictly inside `User` setters; you may add a helper `IsValidContactNumber` that checks numeric-only and length requirements (e.g., Philippines typical mobile format 09XXXXXXXX length 11) and enforce it at registration.
 5. **Cart & Transaction limits**
 - Cart: max 10 distinct items (enforced by `Cart::AddItem` / `BuyerActions` logic).
 - Transaction: max 5 items per transaction (enforced by `Transaction::AddItem` and the checkout flow which groups items by seller).
 6. **Concurrency / atomicity**
 - This is a single-threaded CLI. No concurrency controls for files. If multiple processes edit the same `data/` files concurrently, you may see inconsistencies.
-

Where to extend / recommended tasks (if you want to polish the project)

- Add `DeleteUser`, `DeleteItem`, `DeleteTransaction` routines and menu options with optional cascade or warning prompts.
 - Implement stricter input validation when registering a user (length checks, contact number format).
 - Improve transaction file format to be structured (JSON or CSV) if machine-parsable format desired.
 - Add unit tests for core logic (`Cart`, `Transaction` creation, `FileHandling` parsing).
 - Add backups or cart history feature (timestamped snapshots) if you want to preserve previous carts.
 - Implement logging (to file) for diagnostics and auditability.
 - Add confirmation prompts for deletes and irreversible actions.
-

Quick reference: important constants and where to change them

- Cart filename extension and path: built in `BuyerMenu::Show()` as `"data/carts/" + std::to_string(userId) + ".bag"`. Change `".bag"` to `".txt"` if you prefer.
 - Column spacing (table printing)
 - `admin_actions.cpp` defines static const `size_t COLUMN_SPACING = <value>` for admin tables.
 - `utils.cpp` defines static const `size_t COLUMN_SPACING = <value>` for item tables.Change these constants to adjust spacing between columns globally for each table type.
 - Admin password default is `"H3LLo?"` set in `AdminActions` constructor. Change in `AdminActions` if needed.
-

Example (copyable) — sample User registration flow call sequence

1. User chooses 1. `Register as a User` in `MainMenu`.
2. `MainMenu` reads input and constructs:
3. `User u(userId, password, name, address, contact);`
4. `bool ok = app_.RegisterUser(u);`
5. `ShoppingApp::RegisterUser` checks:
 - `users_.size() < 100`
 - `userID` uniqueness

- If passes, `users_.push_back(u);` and returns `true`.
-